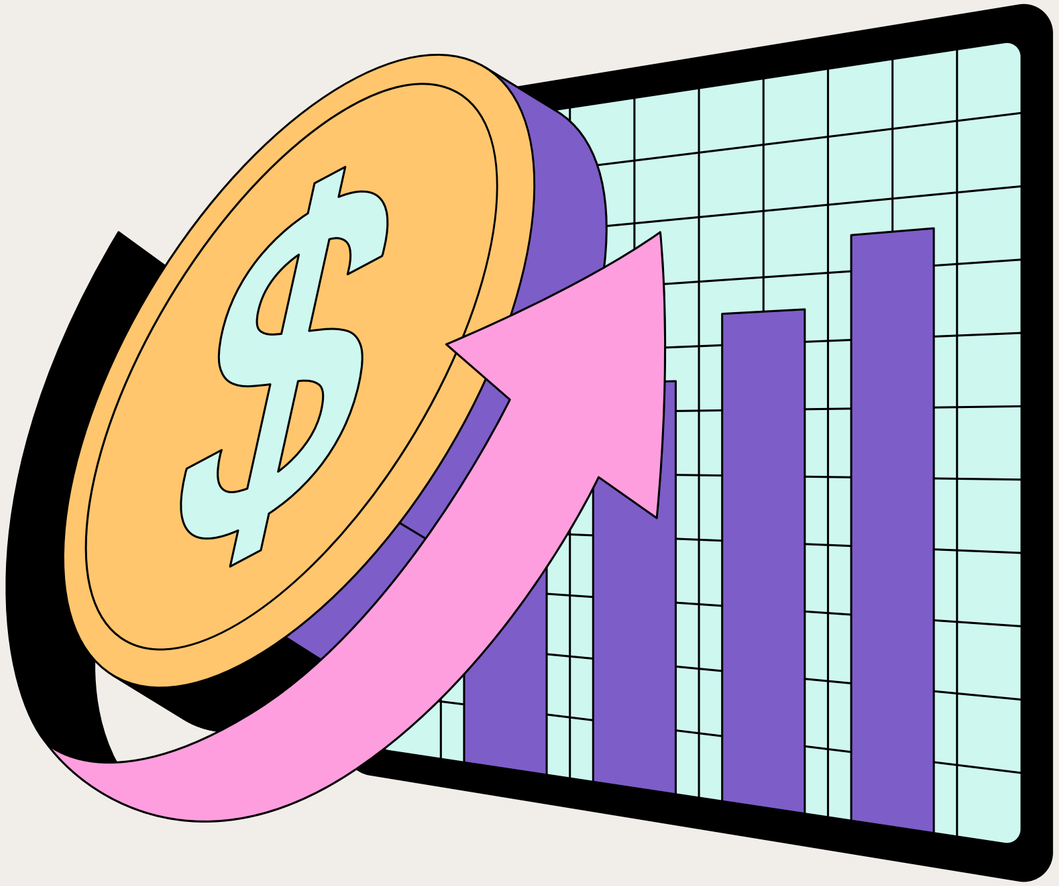
DSA Project

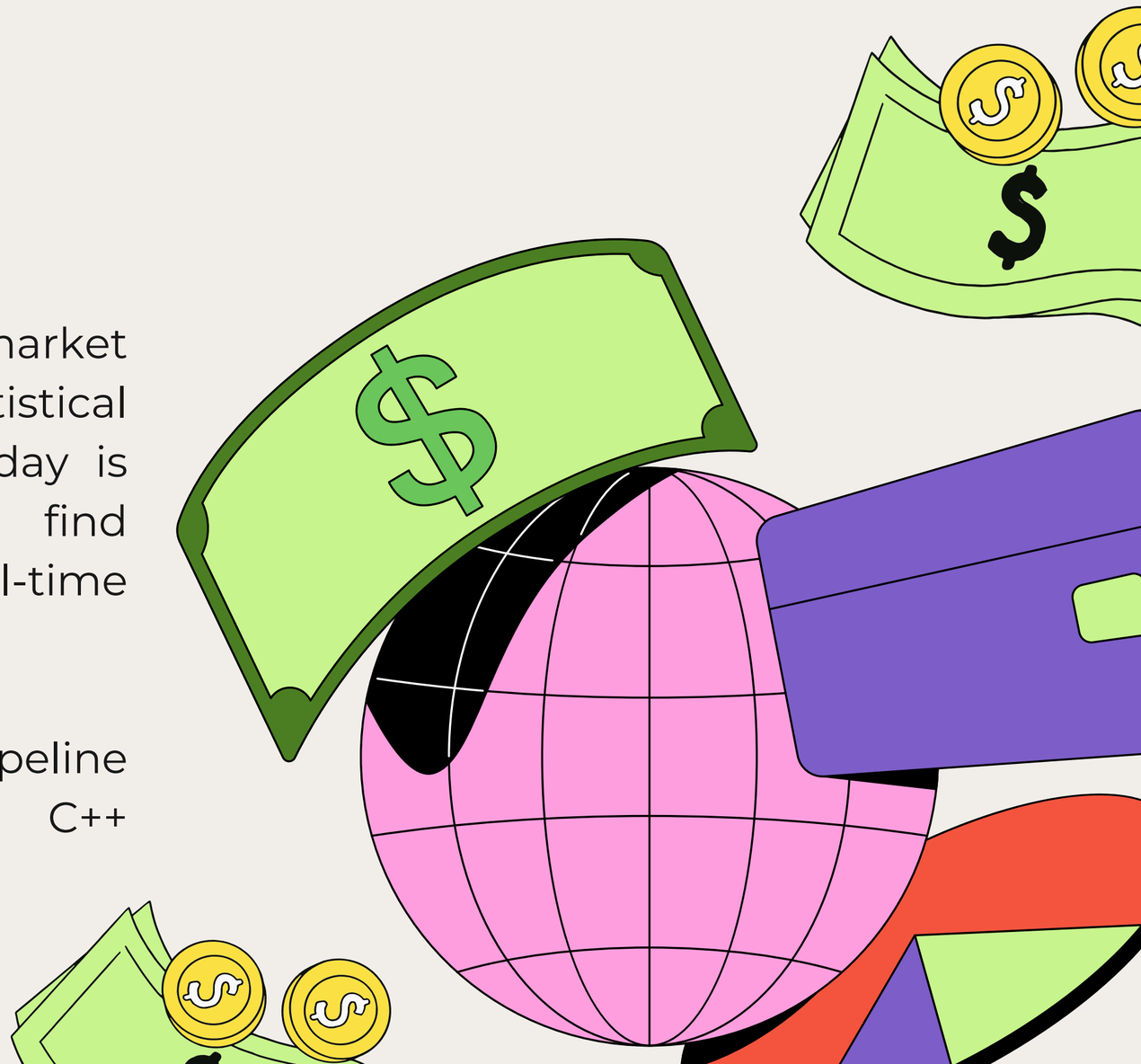
Stock Analyzer



Introduction

- **Problem Statement:** Analyzing live stock market data traditionally requires heavy, slow statistical libraries. Re-scanning historical data day-by-day is highly inefficient, making it difficult to find continuous trends and extremes in a real-time stream.

- **Our Solution:** We built a decoupled data pipeline using a Python data-fetcher and a C++ computational engine.



Core Architecture – Stock Span (Stack)

- Concept: Calculates consecutive days a stock's price has been rising or staying the same.
- How it Works: Instead of checking every past day, a Stack pops smaller previous prices and absorbs their span.

```cpp
int span = 1;
while (!spanStack.empty() && spanStack.top().first <= price) {
    span += spanStack.top().second; // Absorb previous spans
    spanStack.pop();
}
spanStack.push({price, span});
```

Core Architecture – Extremes Tracking (Priority Queues)

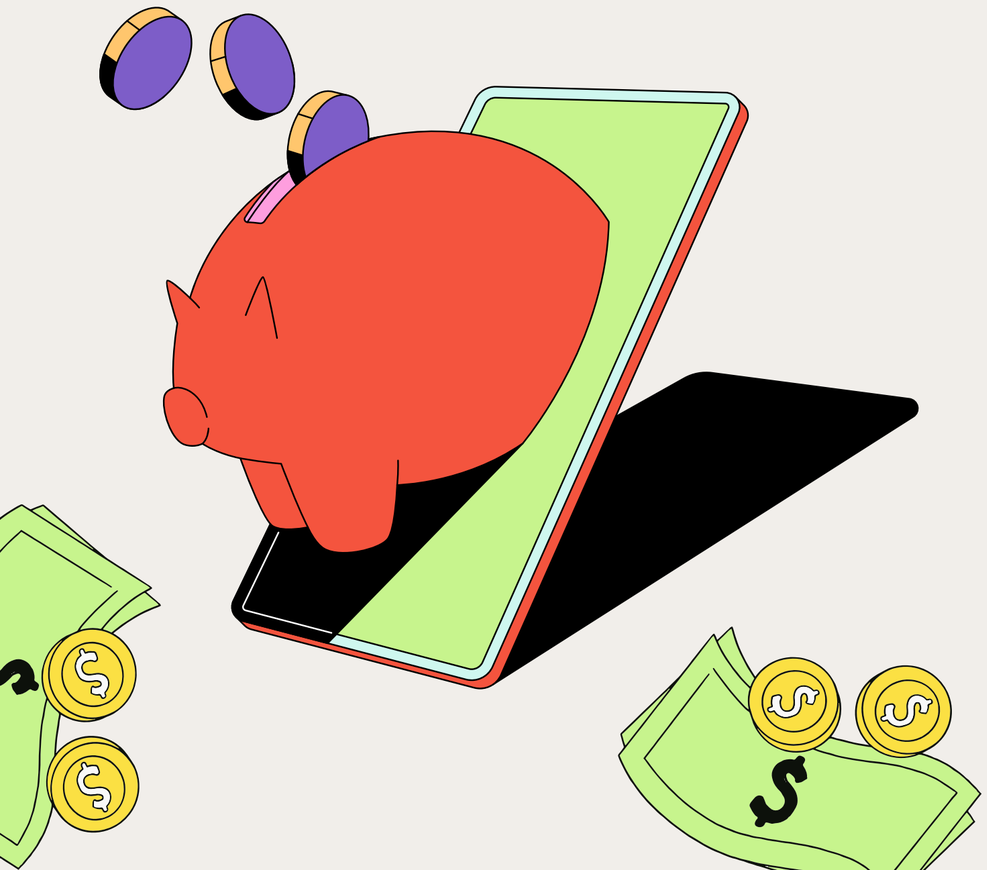
- Concept: Instantly identifying the absolute Highest and Lowest stock prices hit up to the current moment.

- How it Works: We maintain a Min-Heap and a Max-Heap. As every new price streams in, it is pushed into both heaps, providing O(1) access to the extremes without looping through the array.

```
// Pushing current day's price into both heaps
maxHeap.push(price);
minHeap.push(price);

// Getting the extremes instantly
double highest = maxHeap.top();
double lowest = minHeap.top();
```

Core Architecture – Historical Analysis (Merge Sort)



- Concept: Organizing all the collected prices from lowest to highest.
- How it Works: Uses a Divide and Conquer approach to split the array in halves, sort them, and merge them back together. Once sorted, we can easily find the exact percentiles of the stock.

```
void mergeSort(vector<double>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);        // Sort left half
    mergeSort(arr, mid + 1, right);   // Sort right half
    merge(arr, left, mid, right);     // Merge them together
}
```

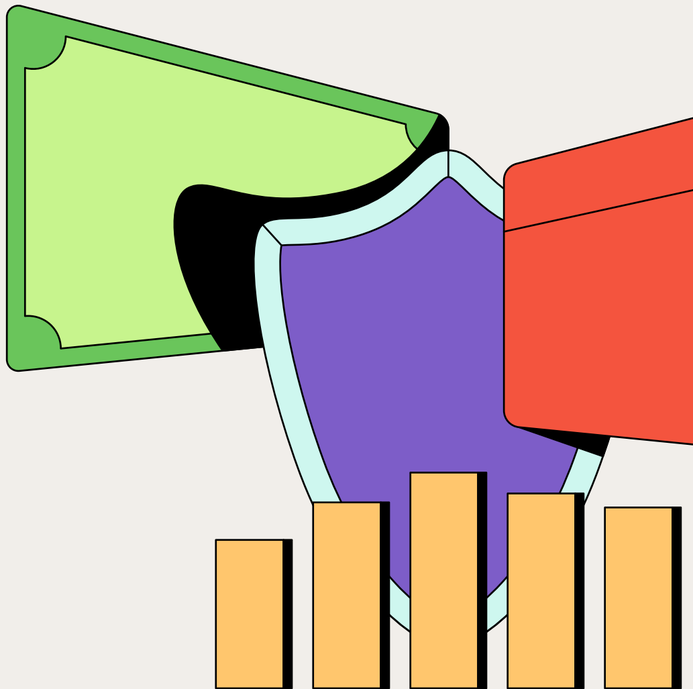
Core Architecture – NGE (Stack)

```
for(int i=n-1; i>=0; i--){
        while(!st.empty() && st.top() <= nums[i]){
            st.pop();
        }

        result[i] = st.empty() ? -1.0 : st.top();
        st.push(nums[i]);
    }
    return result;
}
```

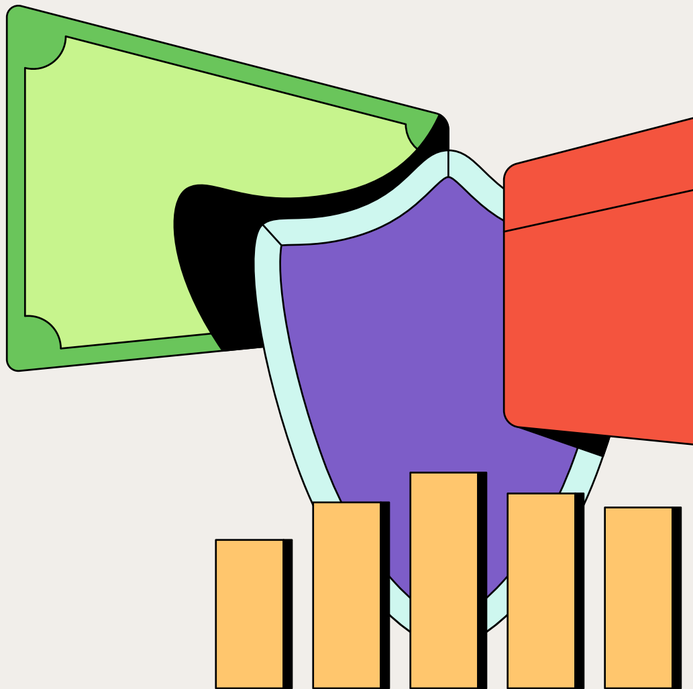
The Streamlit Dashboard

- Live Price Metrics: Displays the all-time high and all-time low on the screen.
  - Powered by: The Min/Max Heaps (getHeapMin(), getHeapMax()).
- Dual-Axis Trend Chart: Plots the daily price line graph heavily overlayed with a bar chart showing rising momentum.
  - Powered by: The Stack-based Stock Span algorithm (getSpan()).
- Historical Quartiles Panel: Displays the overall Median, 25th, and 75th percentiles to check if the stock is overall cheap or expensive.
  - Powered by: The Merge Sort output (getSortedPrices()).
- Live Simulation Tab: Streams data step-by-step automatically, forcing the C++ engine to recalculate all of the above features live on the screen.



Contribution

STOCK SPAN (STACK)	SHUBHASHISH
MERGE SORT AND GREEDY	MOHIT
NEXT GREATER ELEMENT	ADESH
MIN-MAX USING HEAPS	NIKHIL SAINI

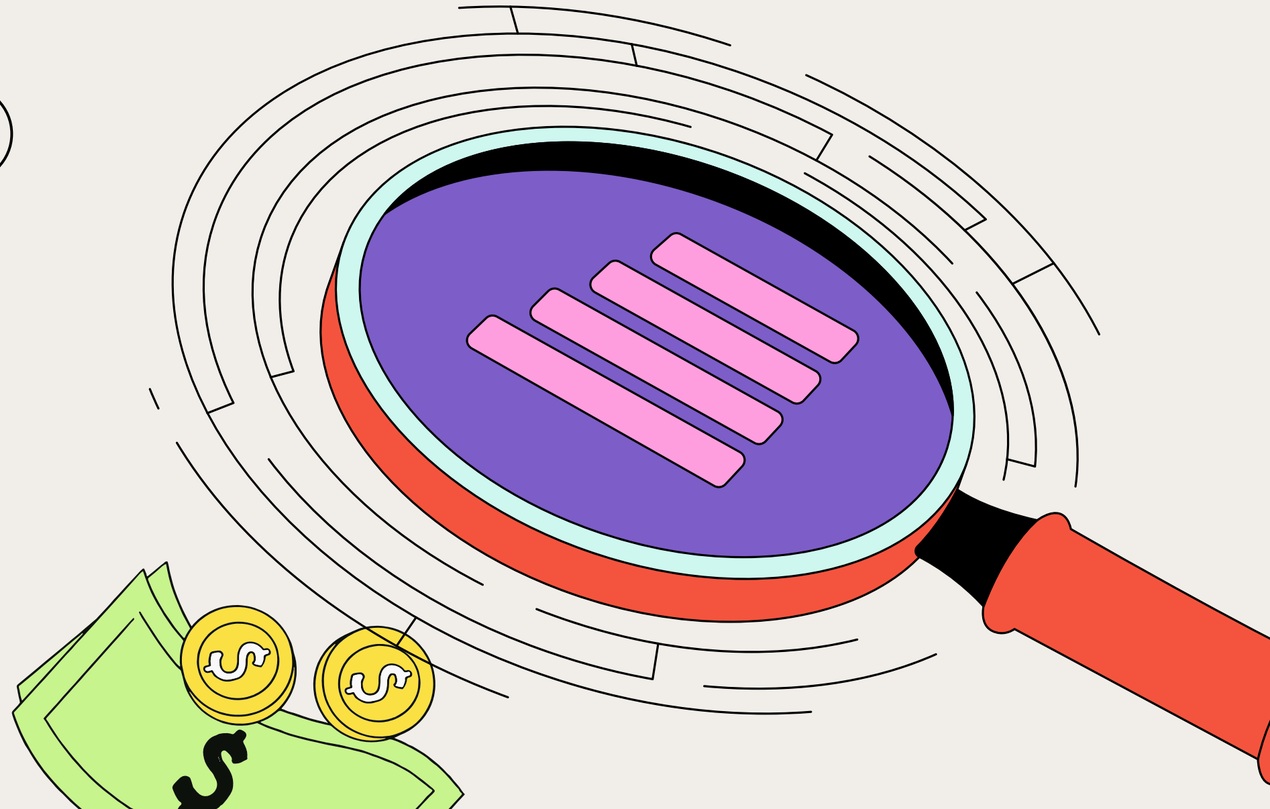


Links & Sources

https://github.com/forbiddenscholar/real_time_stock_analyzer

https://stock-analyzer-dsapro.streamlit.app/

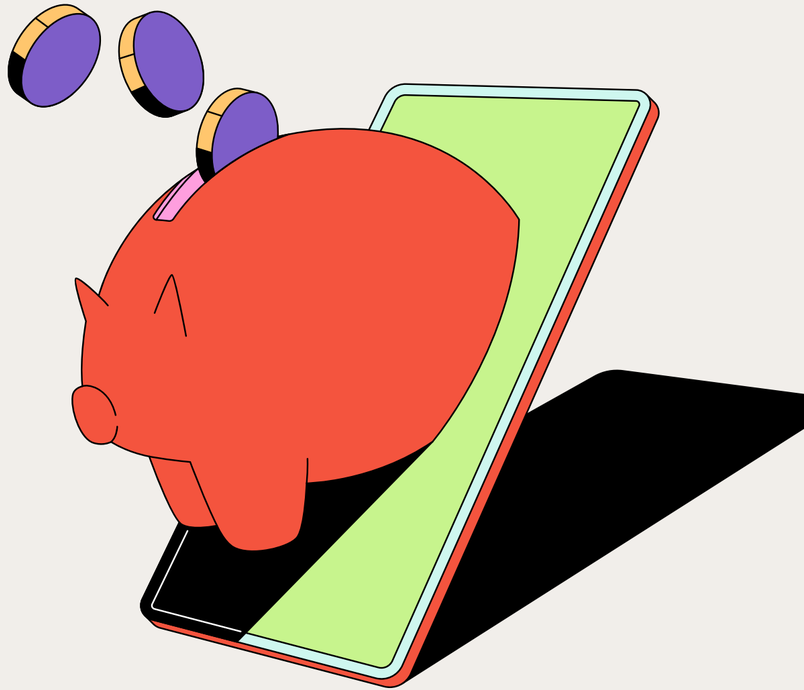
Data Source: yahoo Finance



Thank you !

Shubhashish kumar

Mohit Deharkar

Adesh Singh

Nikhil Saini